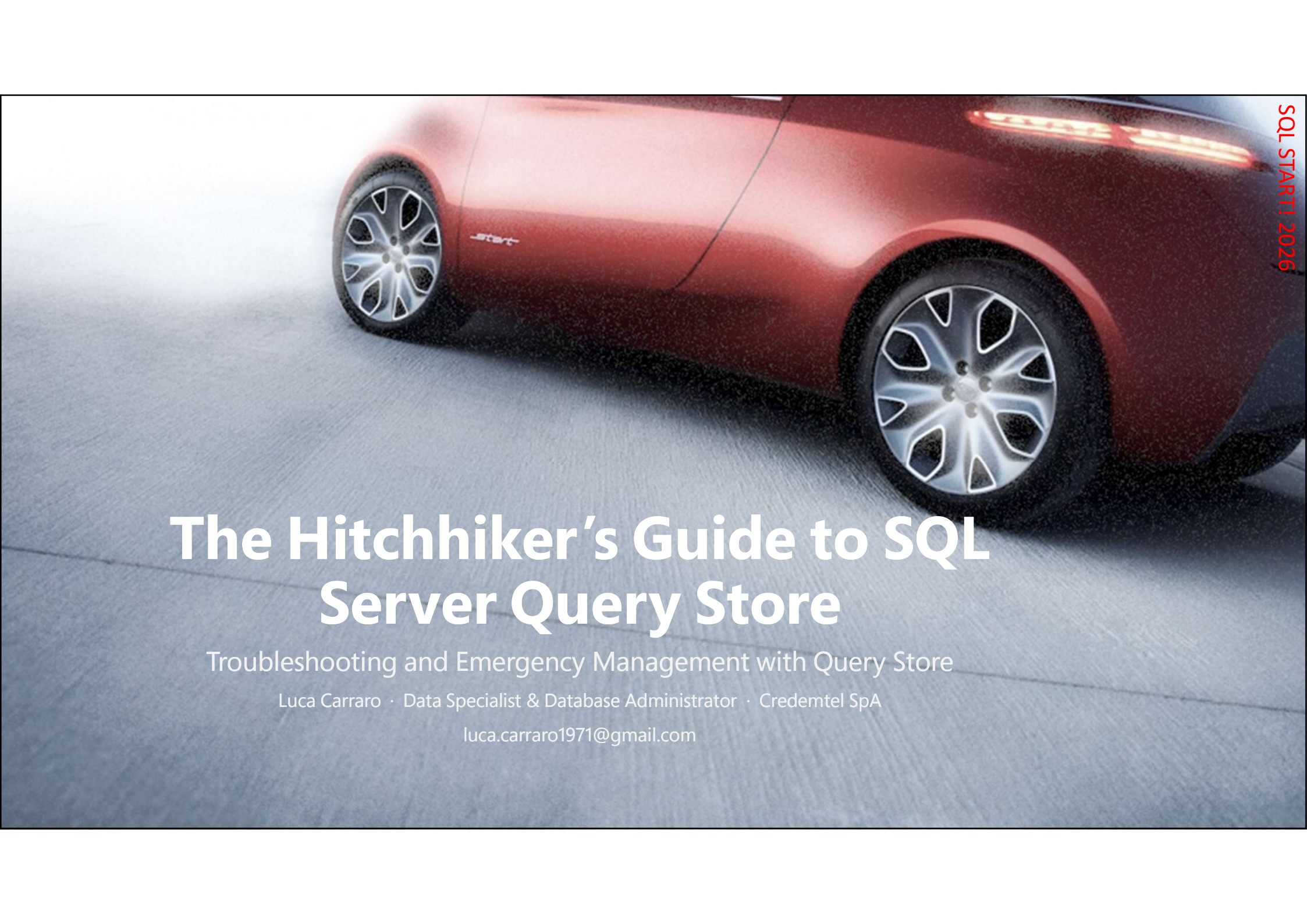




# The Hitchhiker's Guide to SQL Server Query Store

Troubleshooting and Emergency Management with Query Store

Luca Carraro · Data Specialist & Database Administrator · Credemtel SpA

[luca.carraro1971@gmail.com](mailto:luca.carraro1971@gmail.com)

# Sponsors & Organizers



UNIVERSITÀ  
POLITECNICA  
DELLE MARCHE



**Luca Carraro**

Data Specialist &  
Database Administrator

---

*"Life is a Perpetual Beta"*

 [linkedin.com/in/lucacarraro](https://linkedin.com/in/lucacarraro)  
 [github.com/lucacarraro1971](https://github.com/lucacarraro1971)

# About Me

- Messing up with databases since MS SQL Server 2000
- My current playground: High-concurrency SQL Server VLDBs behind DMS
- Extensive experience in delivering IT platforms for the Food & Beverage industry
- Focus: performance tuning and database DevOps
- ✓ Confirmed: I know where my towel is at



CREDEMTel



**DON'T  
PANIC**

# Agenda

**01**

## The Forensic Detective

Telemetry · Parameter sniffing · Plan regressions · Waits · Resource exhaustion

**02**

## The First Responder

Emergency triage · Plan Forcing · Best practices · Edge cases · Hidden traps

**03**

## Code-Free Mind Tricks

Query Store Hints · RECOMPILE · MAXDOP · No app code changes needed

# Query Store or The SQL Server Marvin

SQL Server 2016+ · On by default for NEW databases in SQL Server 2022 · Lives inside the user database

"I have a pain in all  
my queries."



## Query Catalog

Query text, parameterised form, and every execution plan the optimizer generated for it.



## Runtime Stats

Aggregated per interval: CPU, duration, logical I/O, memory grants, and wait categories.



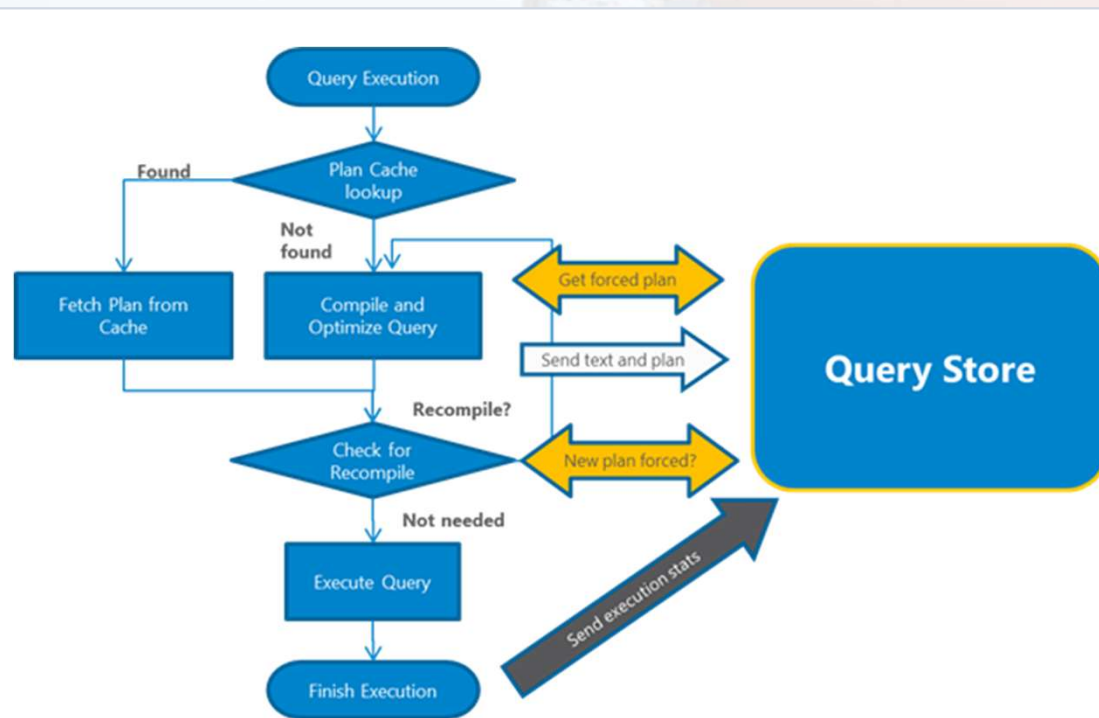
## Plan Management

Force a plan or inject hints — without touching a single line of application code.

**Overhead:** low, but not zero. QUERY\_CAPTURE\_MODE = AUTO (the default since 2019) deliberately skips trivial and infrequent queries to keep it that way. Set ALL **ONLY** for demos and active troubleshooting.

# Query Store is (Mostly) Harmless

*Query Store is not a passive observer: it is read AND written on the compile path, and written again at every execution*



Source: Microsoft Learn

Aggregation per interval is what bounds the cost: 10 000 executions of one plan become ONE row, updated in place.  
An ad-hoc workload breaks that assumption — and that is where "mostly" runs out.

## Production settings

|                             | Default (2019+)  | Erin Stellato · sqlskills.com   |
|-----------------------------|------------------|---------------------------------|
| OPERATION_MODE              | READ_WRITE       | READ_WRITE                      |
| QUERY_CAPTURE_MODE          | AUTO · ALL ≤2017 | AUTO - CUSTOM if ad-hoc (2019+) |
| MAX_STORAGE_SIZE_MB         | 1000 · 100 ≤2017 | 2048 to start · 10240 ceiling   |
| INTERVAL_LENGTH_MINUTES     | 60               | 30 (60 if space-tight)          |
| DATA_FLUSH_INTERVAL_SECONDS | 900              | 900 — leave the default         |
| STALE_QUERY_THRESHOLD_DAYS  | 30               | ≥ 30, match deploy cadence      |
| MAX_PLANS_PER_QUERY         | 200              | 200 (default)                   |
| SIZE_BASED_CLEANUP_MODE     | AUTO             | AUTO — never OFF                |
| WAIT_STATS_CAPTURE_MODE     | ON               | ON                              |

⚠ Minimum patch level: 2016 SP2 CU15 · 2017 CU23 · 2019 CU9

## Watch these waits

QDS\_LOADADB · QDS\_STORE\_SPACE\_MGMT · QDS\_ASYNC\_QUEUE  
spinlocks: QUERY\_STORE\_ASYNC\_PERSIST, SPL\_QUERY\_STORE\_STATS\_COOKIE\_CACHE

**Trace Flag 7745**

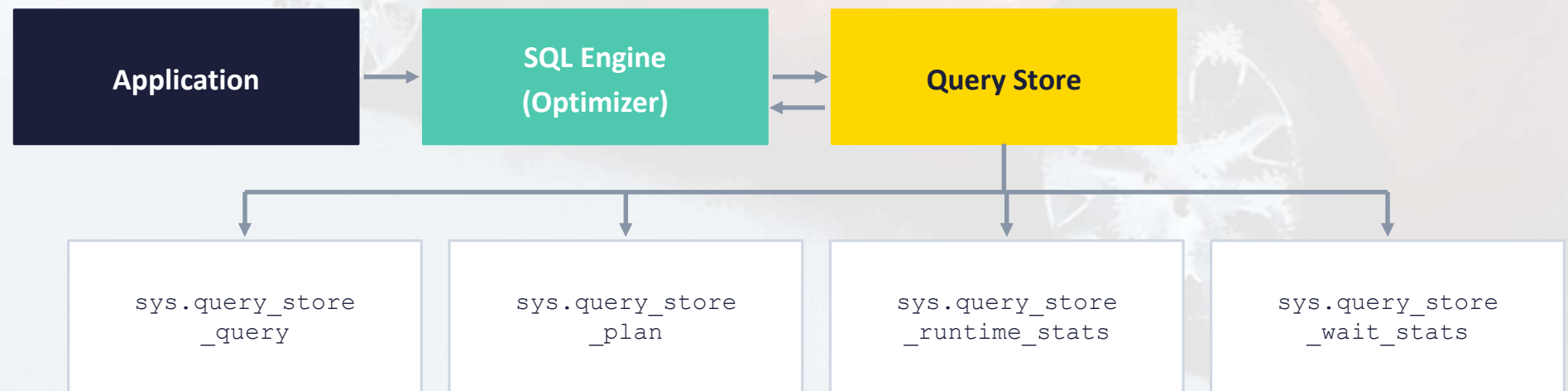
no flush at shutdown: faster failover, loses unflushed data

**Trace Flag 7752**

async load at startup — DEFAULT from SQL Server 2019

# The Flight Recorder

*Query Store captures a time-series of execution metrics — so you can look back in time*



## Key insight:

Query Store keeps a TIME-SERIES of metrics per plan, per aggregation interval — persisted in the user database, so it survives restarts and failovers.

That is what lets you answer, definitively: "Was this query slow BEFORE last Tuesday's deployment?"

PART 1

# The Forensic Detective

---

*"He felt that his whole life was some kind of dream — and he sometimes wondered whose it was."*

— Douglas Adams



# Reading the Flight Recorder

*Starting point: which queries are hurting you right now?*

## SSMS Built-in Reports

- Top Resource Consuming Queries
- Regressed Queries
- Overall Resource Consumption
- Queries with Forced Plans
- Queries with High Variation

## Key Metrics to Sort By

|               |                                             |
|---------------|---------------------------------------------|
| Total CPU     | Biggest CPU offenders over a time range     |
| Avg Duration  | Consistently slow queries                   |
| Exec Count    | High-frequency: small gains, large impact   |
| Memory Grants | Sort/hash spills killing TempDB             |
| Wait Stats    | What is blocking — locks, I/O, parallelism? |

# Parameter Sniffing: The Silent Killer

*One cached plan. Two wildly different parameter values. `SELECT * FROM Orders WHERE CustomerID = @id`*

**Compiled for @id = 1 (500 000 rows)**

Clustered Index Scan

Correct: a seek + 500k key lookups  
would be far worse

✓ 200 ms



**Reused for @id = 9999 (3 rows)**

Same Clustered Index Scan

Should have been Index Seek  
+ 3 key lookups

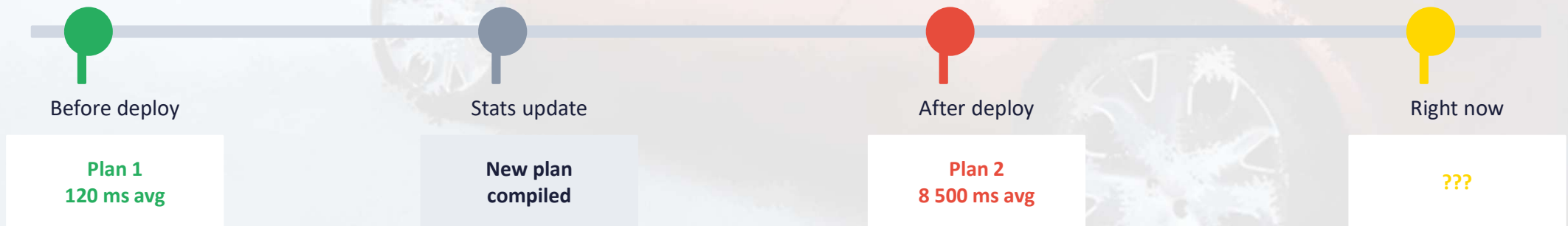
✗ 45 000 ms

## How to diagnose it with Query Store

Queries with High Variation → find ONE query\_id whose duration varies wildly across executions of the SAME plan\_id.

That is the tell: plan regression means several plan\_ids; sniffing means ONE plan\_id behaving inconsistently.

# Plan Regressions: Was It Always This Bad?



```
-- Regression = ONE query, SEVERAL plans, wildly different averages
```

```
WITH pp AS (  
    SELECT p.query_id, p.plan_id,  
           SUM(rs.avg_duration * rs.count_executions)  
             / NULLIF(SUM(rs.count_executions),0) / 1000.0 AS avg_ms  
    FROM sys.query_store_plan p  
    JOIN sys.query_store_runtime_stats rs ON rs.plan_id = p.plan_id  
    GROUP BY p.query_id, p.plan_id)  
SELECT TOP 10 query_id, MIN(avg_ms) AS best_ms, MAX(avg_ms) AS worst_ms,  
              MAX(avg_ms) / NULLIF(MIN(avg_ms),0) AS regression_factor  
FROM pp GROUP BY query_id HAVING COUNT(*) > 1  
ORDER BY regression_factor DESC;
```

# Blocking Waits: Who Is Waiting, and Why?

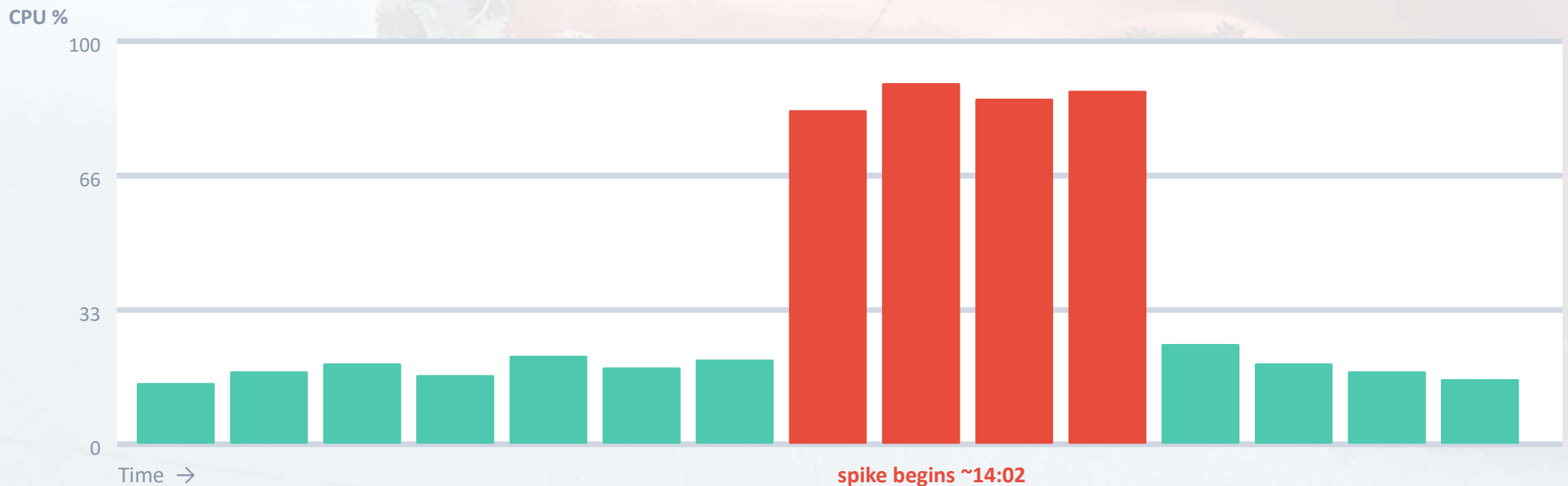
SQL Server 2017+ · `sys.query_store_wait_stats` · Wait time attributed per query, per plan, per interval

| wait_category_desc | rolls up                                                                | what it usually means                                  |
|--------------------|-------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Lock</b>        | <code>LCK_M_S</code> , <code>LCK_M_X</code> , <code>LCK_M_U</code> ...  | Blocking: long transactions, scans taking wide locks   |
| <b>Buffer IO</b>   | <code>PAGEIOLATCH_SH</code> / <code>_EX</code>                          | Reading from disk: cold cache, big scans, slow storage |
| <b>Memory</b>      | <code>RESOURCE_SEMAPHORE</code> , <code>CMEMTHREAD</code>               | Grant pressure: oversized sorts and hashes, spills     |
| <b>Parallelism</b> | <code>CXPACKET</code> , <code>CXCONSUMER</code> , <code>EXCHANGE</code> | Thread imbalance, DOP too high, skewed data            |
| <b>CPU</b>         | <code>SOS_SCHEDULER_YIELD</code>                                        | Real CPU pressure — or a scan burning cycles           |

## Watch out:

Query Store aggregates individual wait types into ~23 categories. You give up `sys.dm_os_wait_stats` granularity — but you gain per-query history you can look back at.

# Resource Exhaustion: The Sudden Spike



## Investigative workflow:

1. Note the spike time → 2. Filter QS to that window → 3. Sort by Total CPU → 4. Click the query, compare plans → 5. Check if a new plan was compiled at that exact moment

PART 2

# The First Responder

---

*"A towel is about the most massively useful thing an interstellar hitchhiker can have."*

— Douglas Adams



# Emergency Triage: The "Don't Panic" Checklist

01

## Identify the offending query

Query Store → Top Resource Consumers. Filter last 30–60 min. Sort by Total Duration or CPU.

02

## Confirm the plan changed

Click the query → Plan Summary. Multiple plans? Compare the good plan vs. the current bad plan.

03

## Verify causation, not correlation

Check `sys.query_store_runtime_stats`: did `avg_duration` spike exactly when the new plan was compiled?

04

## Force the known-good plan

Right-click the good plan in SSMS → Force Plan. Verify response time returns to baseline.

05

## Document and fix root cause

Why did the plan change? Stats update? New index? Parameter change? Schedule a permanent fix.

# Plan Forcing: The "Don't Panic" Button

## How to Force a Plan

### Via SSMS (GUI)

Query Store → select query → Plan Summary tab  
→ right-click plan → Force Plan

### Via T-SQL

```
EXEC sys.sp_query_store_force_plan  
    @query_id = 42, @plan_id = 7;
```

### To roll it back

```
EXEC sys.sp_query_store_unforce_plan  
    @query_id = 42, @plan_id = 7;
```

## What Actually Happens

### plan\_forcing\_type\_desc

Set to MANUAL in sys.query\_store\_plan

### Next execution

Optimizer ATTEMPTS to reproduce it — not guaranteed

### force\_failure\_count

Increments when it cannot; query compiles normally

### is\_forced\_plan

Stays 1 even after a failure — trust the count, not this

### Scope

Per database — survives restart and failover

# Plan Forcing: Edge Cases & Hidden Traps

## Forcing Fails Silently — and Still Looks Forced

HIGH

If the plan can no longer be produced (an index it uses was dropped, the schema changed), SQL Server just compiles normally. `is_forced_plan` stays 1, so it LOOKS fine. Alert on `force_failure_count` and read `last_force_failure_reason_desc`.

## Data Distribution Drift

MED

The plan that was optimal six months ago may be wrong today. Forced plans do not age gracefully — put a recurring review in the calendar, not in your memory.

## It Treats the Symptom, Not the Disease

MED

A stats update is the most common trigger. Force the plan to stop the bleeding, then fix the real cause — indexing, cardinality estimates, or the query itself.

## Cardinality Estimator and Version Changes

LOW

Forced plans persist across restarts and compatibility-level changes, but a plan captured under a different CE model may no longer be reproducible. Re-validate after any major upgrade.

PART 3

# Code-Free Mind Tricks

---

*"Query Store Hints let you change the optimizer's mind without changing a line of code."*

— SQL Server 2022+ · Azure SQL DB / MI



# Query Store Hints

*Inject a query hint into the engine — without touching application code*

## Plan Forcing

- Forces a SPECIFIC plan shape
- Brittle: breaks on schema changes
- Use when: a known-good plan exists
- Available: SQL Server 2016+
- Scope: per query\_id + plan\_id

vs

## Query Store Hints

- Injects a HINT, optimizer picks plan
- More flexible and less brittle
- Use when: behaviour change needed
- SQL Server 2022+ · Azure SQL DB / MI
- Scope: per query\_id only

# QS Hint: OPTION (RECOMPILE)

*Surgical fix for parameter sniffing — no stored procedure changes required*

**The problem:** usp\_GetOrdersByCustomer was sniffed for a huge customer and performs terribly for small ones. It is vendor code — you cannot change it.

```
-- 1. Find the query_id. NB: query_sql_text holds the STATEMENT, not the proc name.
SELECT q.query_id, qt.query_sql_text
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON qt.query_text_id = q.query_text_id
WHERE q.object_id = OBJECT_ID('dbo.usp_GetOrdersByCustomer'); -- 3. Remove it when done

-- 2. Apply the hint
EXEC sys.sp_query_store_set_hints
    @query_id = 127,
    @query_hints = N'OPTION (RECOMPILE)';

-- 3. Remove it when done
EXEC sys.sp_query_store_clear_hints
    @query_id = 127;
```

**Result:** Every execution recompiles against the actual parameter value. Overhead is negligible for low-frequency queries — but measure before applying RECOMPILE to something running thousands of times a minute.

# QS Hint: OPTION (MAXDOP N)

*Throttle parallelism on a per-query basis — no server-wide MAXDOP change needed*

## The problem:

A nightly reporting query grabs 32 threads (MAXDOP 0). During business hours it causes CXPACKET waits and starves OLTP queries. You cannot change the report code.

```
-- Cap this query to 4 threads — no instance-wide change, no app code touch  
  
EXEC sys.sp_query_store_set_hints  
    @query_id      = 88,  
    @query_hints = N'OPTION (MAXDOP 4)';
```

## Other useful hints you can inject:

- OPTION (OPTIMIZE FOR UNKNOWN) — flattens sniffing without per-execution recompile cost, for 2022+ use PSP Optimization if available
- OPTION (USE HINT ('QUERY\_OPTIMIZER\_COMPATIBILITY\_LEVEL\_150')) — To execute the query with a different compatibility level than the database
- OPTION (HASH JOIN) / (LOOP JOIN) — constrain join strategy; aggressive, for edge-case scenarios, so test before you ship it



**DON'T  
PANIC**

# It's demo time!

*"I love deadlines. I love the whooshing sound they make as they fly by."*

— Douglas Adams

# Key Takeaways

- 01 Query Store is your flight recorder. Turn it on. Leave it on. Cost: minimal. Value: priceless.
- 02 When something breaks on Friday afternoon — don't panic. Open Query Store. Filter the time window. Sort by CPU or Duration.
- 03 Plan Forcing is a valid emergency stabiliser — but it is NOT a permanent fix. Find the root cause.
- 04 Query Store Hints (SQL Server 2022+) inject surgical fixes without touching application code. Use them wisely.
- 05 Monitor `force_failure_count`. A silently dropped forced plan is worse than no forced plan at all.

# Essential Resources

## Official Docs

- [learn.microsoft.com](https://learn.microsoft.com) → Monitor performance by using the Query Store
- [learn.microsoft.com](https://learn.microsoft.com) → Best Practices for Monitoring Workloads with Query Store

## Community Deep Dives

- Erin Stellato — Query Store Settings / Best Practices · [sqlskills.com/blogs/erin](https://sqlskills.com/blogs/erin)
- UGISS — Italian SQL Server User Group · [ugiss.org](https://ugiss.org)

## Tools

- `sp_QuickieStore` (Erik Darling) — [github.com/erikdarlingdata/DarlingData](https://github.com/erikdarlingdata/DarlingData)
- Brent Ozar First Responder Kit — `sp_BlitzCache`, `sp_BlitzQueryStore`

# So Long, and Thanks for All the Fish.

## Questions?

**Luca Carraro**

Data Specialist & Database Administrator · Credemtel SpA

[linkedin.com/in/lucacarraro](https://www.linkedin.com/in/lucacarraro)

[luca.carraro1971@gmail.com](mailto:luca.carraro1971@gmail.com)

Slides + demo script:

[github.com/lucacarraro1971/Presentazioni/tree/main/SQLSTART2026](https://github.com/lucacarraro1971/Presentazioni/tree/main/SQLSTART2026)

SQL Start! 2026 · Ancona